Zeroth Review — Complete Study Guide

Project: Embodied Semantic Asset Recall Robot
Team: Sidharth Sajith, Nidhi, Vasudev, Jibin
Presentation: Tomorrow
Format: This document covers every slide + deep-dive on literature survey + Q&A prep

---

PART 1: SLIDE-BY-SLIDE BREAKDOWN

Slide 1 — Title

"Embodied Semantic Asset Recall Robot"
Offline Industrial Object Search, Recall, and Verification via Edge-Optimized Vision-Language Models

What to say:

> Our project is an autonomous mobile robot that remembers where objects are in a lab/workshop, and finds them again — even if moved — using only low-cost hardware and fully offline AI.

Key terms to know:
- **Embodied** = physically exists in the world (not just software)
- **Semantic** = understands meaning (not just geometry)
- **Asset Recall** = remembers and locates objects
- **Edge-Optimized VLMs** = Vision-Language Models running locally on the robot

---

Slide 2 — Motivation

Problem: In shared labs/workshops, tools get misplaced constantly.

Why existing solutions fail:
- CCTV — records video but can't answer "Where is the multimeter?"
- RFID/Barcode — requires tagging every item
- Manual inventory — slow, infrequent, error-prone

Key stat to mention: McKinsey (2022) — manufacturing workers spend up to **25% of time** searching for tools.

Why this matters for safety: Fire extinguishers, first-aid kits must be locatable during emergencies.

Differentiator from cloud robots: Privacy + no internet dependency.

Slide 3 — Problem Statement

4 core functions:
1. Build semantic memory of objects + locations
2. Recall where an asset was last seen
3. Navigate + visually verify
4. Intelligent search when moved/missing

3 constraints (emphasize these):
- Fully offline — no cloud AI

- Low-cost — no LiDAR, no depth cameras
- Edge-only — all AI on Jetson 8 GB

Scope: Single room (5m x 5m), 10-15 object classes, voice/text queries.

Slide 4 — Existing Solutions

Approach	Limitation
CCTV	Passive recording, no queries
RFID/Barcode	Needs tagging discipline
Manual inventory	Slow, infrequent
Cloud robots (Spot)	>$75K, needs internet
Research prototypes (DovSG, ConceptGraphs)	Need depth cameras, cloud LLMs, high-end GPUs

Gap: No existing low-cost, fully offline system combining semantic memory + moved-object search.

Slide 5 — Literature Survey (EXPANDED — see Part 2 below for full detail)

We surveyed **25+ papers** across 5 research areas:
1. **Semantic Mapping** — VLMaps, CLIP-Fields, ConceptFusion
2. **Scene Graphs** — ConceptGraphs, DovSG, HOV-SG
3. **Dynamic Navigation** — OpenObject-NAV, OpenIN
4. **Probabilistic Search** — ZAVIS, SEEK, L3MVN
5. **Industrial Inspection** — AutoInspect, Simbe Tally, Spot

10 key papers on the slide (see Part 2 for detailed breakdown of each).

Common thread: Each solves part of our problem, but **none** integrates persistent memory + moved-object search + edge-only operation.

Slide 6 — Research Gap

Venn diagram concept (5 circles):
- Semantic Memory ✓ — ConceptGraphs, DovSG
- Dynamic Object Tracking ✓ — DovSG, OpenObject-NAV
- Probabilistic Search ✓ — ZAVIS, SEEK
- Edge/Offline Operation ✓ — YOLO-World, Florence-2
- Industrial Lab Domain ✓ — AutoInspect, Spot

Our project = intersection of ALL FIVE. No existing work covers this combined space.

Specific gap: No system provides persistent memory (power-cycle survival) + probabilistic moved-object search + 8 GB edge compute + no depth sensors in a real lab setting.

Slide 7 — Proposed Solution

Six core behaviors (MAP -> REMEMBER -> RECALL -> VERIFY -> SEARCH -> REPORT):

1. **Map** — Patrol + build geometric + semantic map
2. **Remember** — Store object ID, location, timestamp, confidence, nearby objects
3. **Recall** — Query "Where is the multimeter?" -> navigate to last-known
4. **Verify** — Visually confirm with VLM
5. **Search** — Bayesian reasoning when moved
6. **Report** — Dashboard with bounding boxes, heatmap, reasoning log

Key differentiator: "This is NOT just object detection on a robot. The contribution is persistent memory + intelligent search behavior."

Slide 8 — Architecture (9 layers)

Bottom-up:

Layer	Component	What it does
1. Mobility	4WD chassis + ESP32	Motor control, wheel encoders
2. Localisation	Smartphone ARCore + wheel odom + EKF	Pose estimation without LiDAR
3. Mapping	OctoMap (5 cm voxels)	3D occupancy grid
4. Perception	YOLO-World (fast) + VLM (verification)	Two-stage AI pipeline
5. Semantic Memory	JSON scene graph	Persistent object memory
6. Search Engine	Bayesian belief over zones	Moved-object reasoning
7. Navigation	Nav2 MPPI	Path planning, obstacle avoidance
8. Decision Logic	BehaviorTree.CPP + Groot2	Task coordination
9. Interface	Whisper STT + web dashboard	Voice/text + visualization

Two-stage perception is key: Fast detector runs continuously (~5-10 Hz), VLM only triggers at verification stops. Keeps the Jetson responsive.

Slide 9 — Features (8 total)

Feature	One-liner
Semantic Asset Memory	Stores what, where, when, nearby
Natural Language Queries	"Find the multimeter"
Moved-Object Search	Bayesian reasoning, not random

Safety Item Inspection	Check extinguisher, first-aid, helmet
Change Detection	"Multimeter moved from bench to shelf"
Explainable Search	Shows reasoning steps + heatmap
Offline Operation	No internet needed
Open-Vocabulary Detection	No retraining for new objects

Slide 10 — Methodology (4 Phases)

Phase 1 (Weeks 1-12, Jul-Sep 2026) — Hardware foundation
- Build chassis, motor control, power
- Smartphone -> Jetson data pipeline
- ARCore + wheel odometry fusion
- OctoMap generation

Phase 2 (Weeks 13-20, Oct-Nov 2026) — Perception + Memory
- Benchmark detectors on lab objects
- Build semantic scene graph (JSON)
- Basic recall: navigate to last-known location

Phase 3 (Weeks 21-28, Dec 2026-Jan 2027) — Intelligence
- Bayesian search engine
- Behavior tree for search logic
- Compare vs. baselines (random, last-known, nearest-zone)

Phase 4 (Weeks 29-36, Feb-Apr 2027) — Interface + Validation
- Voice commands (Whisper)
- Web dashboard
- Full demos + evaluation

Slide 11 — Hardware & Software Stack

Hardware (total ~Rs. 30,000-35,000):
- Jetson Orin Nano 8 GB (~Rs. 20,000) — main compute
- Android phone (ARCore) — VIO + RGB camera (already owned)
- ESP32 — motor control, encoders
- 4WD chassis + motors + encoders (~Rs. 5,000)
- 4S LiPo battery + regulators (~Rs. 3,000)
- USB hub, mic, mounting (~Rs. 2,000)

Software:
- ROS 2 Humble, Nav2, OctoMap
- YOLO-World (detection), Florence-2/NanoLLaVA (VLM verification)
- Whisper.cpp (speech), BehaviorTree.CPP (logic)
- React web dashboard (visualization)

Slide 12 — Roadmap

When	What
Jul-Aug 2026	Research + procurement (WE ARE HERE)
Sep-Oct 2026	Phase 1: Robot drives, localizes, maps
Nov-Dec 2026	Phase 2: Detects + remembers objects
Jan-Feb 2027	Phase 3: Bayesian search works
Mar 2027	Phase 4: Voice + dashboard
Apr 2027	Final demos + report

Slide 13 — Expected Outcomes

Measurable targets:
- Stationary object recall: >80% success
- Moved-object search: >60% success
- Time-to-find: >2x faster than random
- System runtime: >15 min without crash

3 demos to show:
1. **Asset Recall** — "Find the multimeter" -> navigate + verify
2. **Moved-Object Search** — Move tool -> robot finds it via Bayesian search
3. **Safety Inspection** — "Check safety items" -> report presence/absence

Offline demo: Run with Wi-Fi/Bluetooth disabled.

Slide 14 — Future Scope

- Multi-room/multi-floor
- Multi-robot shared memory
- Robotic arm for retrieval (not just location)
- Learning search priors from experience
- Hospitals, retail, warehouses

Slide 15 — References

15 key references. All accessible on arXiv. Be ready to discuss any of them.

PART 2: LITERATURE SURVEY DEEP DIVE

This is the most likely area for faculty questions. Study this carefully.

Research Area 1: Semantic Mapping

VLMaps (Huang et al., 2022, CoRL)

- **Venue:** Conference on Robot Learning (CoRL)
- **What it does:** Fuses CLIP (language) features into a 3D spatial grid. You can query: "go to the kitchen" or "find the table."
- **How it works:** While the robot moves, it takes RGB frames + depth, encodes each pixel through CLIP, and projects the features into a 3D voxel grid. Each voxel stores a CLIP feature vector. Queries work by finding the voxel whose CLIP feature best matches the text query.
- **Why we chose it:** Foundational work showing language-driven navigation is possible.
- **Key limitation for us:** Dense per-voxel CLIP features are too heavy for 8 GB Jetson. No object-level memory — it stores features, not objects. Static map — no handling of moved objects. Requires RGB-D camera.

CLIP-Fields (Jatavallabhula et al., 2022, RSS)

- **What it does:** Creates a continuous semantic field over space using CLIP embeddings. Like VLMaps but with a learned implicit representation.
- **How it works:** Uses a neural field (NeRF-like) that maps (x,y,z) coordinates to CLIP feature vectors. You can query any point for its semantic meaning.
- **Why we chose it:** Shows alternative approach to semantic mapping.
- **Key limitation:** NeRF-style optimization is slow and offline. No temporal persistence. Requires RGB-D.

ConceptFusion (Jatavallabhula et al., 2023, RSS)

- **What it does:** Open-set multimodal 3D mapping. Fuses vision, language, and audio features into a 3D map.
- **How it works:** For each point in the 3D reconstruction, stores features from CLIP (vision), sentence embeddings (language), and audio features. The map can be queried with text, images, or sounds.
- **Why we chose it:** Shows the value of foundation models in 3D maps.
- **Key limitation:** Offline reconstruction — not real-time. Still RGB-D and cloud-dependent.

Research Area 2: 3D Scene Graphs

ConceptGraphs (Gu et al., 2024, ICRA)

- **Venue:** IEEE International Conference on Robotics and Automation (ICRA) — top robotics venue
- **What it does:** Builds hierarchical 3D scene graphs with open-vocabulary object detection. Graph has rooms, objects, and relationships.
- **How it works:** Uses SAM (Segment Anything) to detect object segments in images, Grounding DINO to label them with text, and projects them into 3D using RGB-D. Creates a graph where nodes = objects, edges = spatial relationships.
- **Why we chose it:** State-of-the-art in open-vocabulary scene graphs. We borrow the concept of object-level memory.
- **Key limitation:** Static reconstruction only — does not handle dynamic updates. Requires RGB-D camera. Uses cloud LLMs (GPT-4) for reasoning.

DovSG — Dynamic Open-Vocabulary Scene Graphs (Zhu et al., 2025, RA-L)

- **Venue:** IEEE Robotics and Automation Letters (RA-L)
- **THIS IS THE CLOSEST WORK TO OURS.**
- **What it does:** Builds dynamic 3D scene graphs that update when objects move. If you move a cup from table to shelf, the graph is locally updated.
- **How it works:** Uses DROID-SLAM for visual odometry, SAM for segmentation, CLIP for open-vocabulary labeling. When an object disappears from its previous location, it runs a local sub-graph update — checks nearby surfaces, re-detects, and updates edges.
- **Why we chose it:** Most relevant existing work. Validates our approach of dynamic object memory. Achieved 35% success on long-horizon tasks (vs 5% for baseline).
- **What we borrow from it:** The carrier-relationship concept ("multimeter ON bench"), local graph updates.
- **Key limitation for us:** Requires RGB-D camera + DROID-SLAM (heavier than ARCore). Uses GPT-4o (cloud) for reasoning. Focused on mobile manipulation (has arm). No probabilistic search behavior. Cloud-dependent.

HOV-SG / OSG Navigator (Werby et al., 2024)

- **What it does:** Hierarchical open-vocabulary scene graphs for navigation. Multi-level: rooms -> objects -> relationships.
- **How it works:** Builds a graph with room-level nodes, object-level nodes, and edges representing spatial/temporal relationships. Uses this for long-horizon navigation tasks.
- **Why we chose it:** Shows scene graphs can be used for navigation.
- **Key limitation:** Heavy compute. Static environment assumption (no moved objects).

Research Area 3: Dynamic Object Navigation

OpenObject-NAV (Li et al., 2024)

- **What it does:** Tracks object displacement through **carrier relationships** — e.g., "the cup WAS ON the table, NOW it is ON the shelf." The carrier relationship (object-on-surface) is tracked.
- **How it works:** When an object moves, it maintains the carrier relationship chain. The robot knows objects don't teleport — they move through carriers (tables, shelves, boxes).
- **Why we chose this:** Directly inspiring our "multimeter moved from bench to shelf" reasoning.
- **Key limitation:** Simulation-only. No real-world deployment. High compute. No persistent memory across sessions.

OpenIN (Li et al., 2025)

- **What it does:** Instance-oriented navigation in dynamic environments. Tracks specific object instances (not just classes).
- **Why we chose it:** Shows instance-level tracking is feasible.
- **Key limitation:** Also simulation-only. Requires depth cameras.

Research Area 4: Probabilistic & Active Search

ZAVIS — Zero-Shot Active Visual Search (Karnan et al., 2022)

- **Venue:** arXiv (later extended to conference)
- **What it does:** Zero-shot object search using language queries. Given a text description, it actively searches for the object by selecting informative viewpoints.
- **How it works:** Uses semantic priors — "keys are likely near the door" — to initialize a belief map. Then selects viewpoints that maximize information gain, moving the camera to look at high-probability areas.
- **Why we chose it:** Strongly influences our Bayesian search design. The semantic prior concept (object class -> likely locations) is directly adopted.
- **Key limitation:** No persistent memory — search restarts from scratch every episode. No temporal persistence.

SEEK — Semantic rEasoning for Object Goal Navigation (Ginting et al., 2024, RSS)

- **Venue:** Robotics: Science and Systems (RSS) — top robotics venue
- **What it does:** Combines semantic reasoning with POMDP (Partially Observable Markov Decision Process) planning for real-robot object-goal navigation.
- **How it works:** Maintains a belief over object locations. Uses semantic knowledge (e.g., fire extinguishers are on walls) as priors. POMDP solver selects actions to reduce uncertainty.
- **Why we chose it:** Validated on real robots. Shows that probabilistic semantic search works in practice.
- **Key limitation:** Targets **static infrastructure** (fire extinguishers on walls, exit signs). Does not handle movable objects. Uses heavier compute than our discrete Bayes approach.

L3MVN — LLM-guided Visual Navigation (Yu et al., 2023, IROS)

- **What it does:** Uses LLMs (GPT-4) to guide frontier selection for object search.
- **How it works:** LLM receives the current scene description and suggests which direction to explore. "You are in a kitchen. To find a cup, check the counter to your left."
- **Why we chose it:** Shows LLMs can guide search (but we choose not to depend on them).
- **Key limitation:** Cloud LLM dependency. No persistent memory. No probabilistic belief update.

AP-VLM (2024)

- **What it does:** VLM-guided active perception — uses a VLM to decide where to look next.
- **Why we chose it:** Shows VLMs can select informative viewpoints.
- **Key limitation:** Manipulation-centric (arm-based). Assumes known object pose.

Research Area 5: Industrial Inspection

AutoInspect (Renz et al., 2024)

- **Venue:** arXiv (ETH Zurich)
- **What it does:** Long-term autonomous industrial inspection robot. Patroles facilities daily, checks fixed infrastructure (pipes, gauges, panels).
- **How it works:** Uses topological maps + LiDAR SLAM. Repeats inspection routes. Detects anomalies in fixed equipment.
- **Why we chose it:** Validates the industrial inspection application. Shows long-term autonomous operation is feasible.

- **Key limitation:** Inspects **fixed infrastructure** — not movable tools/assets. Uses LiDAR (expensive). No semantic memory of objects.

Simbe Tally (Commercial)

- **What it does:** Retail shelf-scanning robot. Checks inventory levels on store shelves.
- **How it works:** RFID + cameras. Patrols aisles, scans shelf labels and tags, reports out-of-stock items.
- **Why we chose it:** Proves commercial value of autonomous visual inventory.
- **Key limitation:** Retail planogram focus. RFID-dependent. Cloud backend. Not open-vocabulary.

Boston Dynamics Spot (Commercial)

- **What it does:** Legged inspection platform for industrial sites.
- **How it works:** Walks through facilities with payload cameras. Can read gauges, detect leaks, map environments.
- **Why we chose it:** Validates market demand for mobile inspection robots.
- **Key limitation:** $75K+ cost. Cloud-dependent for AI. Overkill for lab-scale asset recall.

Research Area 6: Edge AI (The Enablers)

These are the models/tools that make our system possible on Jetson:

Model	What it is	Why we chose it
YOLO-World (CVPR 2024)	Open-vocabulary detector	Real-time (~10 FPS on Jetson), no retraining needed
Florence-2 (Microsoft, 2023)	Lightweight vision foundation model	77M params, strong accuracy, can be quantized to 4-bit
NanoLLaVA (2024)	Sub-1B VLM for edge	Specifically designed for Jetson deployment
Whisper.cpp (OpenAI)	Local speech-to-text	Tiny model runs on CPU, fully offline
AWQ Quantization	Weight compression	Makes 4-bit VLM inference possible on 8 GB

The Synthesis: How Literature Shapes Our Design

Our Design Decision	Inspired By	Why We Changed It
ARCore VIO (not LiDAR)	AutoInspect uses LiDAR, DovSG uses DROID-SLAM	Replace expensive sensors with phone
Two-stage perception	ConceptGraphs/DovSG use single heavy VLM	YOLO-World runs continuously, VLM only at keyframes

JSON scene graph	DovSG/ConceptGraphs use 3D reconstruction	Lighter, human-readable, persistent
Carrier relationships	OpenObject-NAV, DovSG	Simplified ontology, fewer edges
Discrete Bayes filter (not POMDP)	SEEK uses POMDP, ZAVIS uses priors	Faster (<10 ms/update), runs on Jetson
Behavior Tree executive	Standard ROS 2 + DovSG	Explicit failure handling for search states
Offline Whisper (not Riva)	Most works use cloud ASR	Saves 2+ GB GPU RAM

The Research Gap (Memorize This)

> **No existing system integrates:**
> 1. Persistent object-level semantic memory (survives power cycles)
> 2. Bayesian active search for moved objects
> 3. Fully offline operation on sub-Rs. 40k robot
> 4. Without LiDAR or depth cameras
> 5. In a real shared lab/workshop environment

PART 3: FACULTY Q&A PREP

Most Likely Questions

Q: "Is this not just a robot with a camera and YOLO?"

> No. Detection is one component. The core contribution is **persistent semantic memory** that survives reboot and the **Bayesian search engine** that reasons about moved objects. No existing system combines these on edge hardware.

Q: "Why not just use RFID?"

> RFID requires tagging every object and maintaining infrastructure. Our system identifies any visible, untagged object visually — more flexible.

Q: "How does the Bayesian search actually work?"

> Simple concept: The robot maintains a probability for each zone (bench, shelf, toolbox). Priors come from last-seen location, object type, and carrier relationships. When it looks and doesn't find the object, that zone's probability decreases (but not to zero, because vision has false negatives). The robot checks the highest-probability zone next. This continues until the object is found or probabilities are exhausted.

Q: "What if the robot can't find the object?"

> It reports failure after checking all high-probability zones, with a log showing where it looked and why.

Q: "Why offline when everyone has internet?"

> Labs have unreliable WiFi. Privacy regulations restrict sending images to cloud. Offline eliminates latency and subscription costs. Safety-critical recall cannot depend on cloud availability.

Q: "Whats the budget?"

> Approximately Rs. 30,000-35,000, excluding the smartphone (already owned).

Q: "How is this different from DovSG?"

> DovSG requires an RGB-D camera, DROID-SLAM, and GPT-4o (cloud). We use a smartphone camera, ARCore, and on-device models. DovSG has no search behavior — it updates the graph but doesn't actively search for moved objects. Our Bayesian search engine is the key difference.

Q: "DovSG achieved 35% vs 5% — can you match that?"

> We target different metrics. DovSG measures long-horizon task completion. We measure search success rate and time-to-find. Our target is >60% for moved-object search with 2x speedup over random.

Q: "What is the total cost breakdown?"

> Jetson: Rs. 20,000. Chassis + motors: Rs. 5,000. ESP32: Rs. 500. Battery + regulators: Rs. 3,000. Phone mount, hub, mic: Rs. 2,000. Total: ~Rs. 30,000-35,000.

Q: "Can you demo anything tomorrow?"

> We're in the research/procurement phase. We can show our literature survey, architecture design, workspace setup. Hardware assembly begins August.

Key Numbers to Remember

- 25+ papers surveyed
- 5 research areas covered
- Rs. 30,000-35,000 budget
- 8 GB Jetson memory
- 10-15 object classes
- 5m x 5m room

> 80% stationary recall target

> 60% moved-object search target

- 2x speedup target vs random search
- 4 phases over 36 weeks

Good luck with the presentation!